

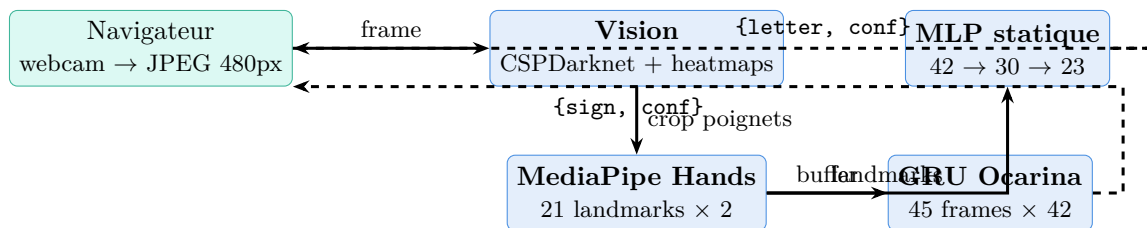
HearMyHands

Flow des données entre les modèles

Équipe HearMyHands — Polytech Nantes

1 Architecture générale

L'application repose sur **trois modèles chaînés** côté backend et un modèle auxiliaire côté client en mode dynamique. Le navigateur envoie ses frames webcam au serveur Flask via Socket.IO en binaire, et le serveur répond en JSON à chaque frame.



Transport. Frames JPEG (~ 480 px, qualité 0,7) en binaire sur Socket.IO. Pipeline avec back-pressure : 4 frames en vol simultanément max, ack par séquence, application uniquement de la plus récente.

2 Mode statique — MLP par frame

Objectif : reconnaître une lettre par frame, fluide en continu. Utilisé pour les lettres statiques de l'alphabet LSF (toutes sauf J, P, X, Z qui demandent un mouvement).

Flux serveur à chaque frame

1. Décodage JPEG reçu.
2. **Vision** : CSPDarknet53 + heatmaps + Soft-Argmax produisent 9 keypoints (cou, épaules, coudes, poignets, hanches).
3. Crop de deux fenêtres centrées sur les poignets ($\rightarrow$ deux sous-images).
4. **MediaPipe Hands** sur chaque crop : 21 landmarks (x, y) par main détectée.
5. **MLP** ($42 \rightarrow 30 \rightarrow 23$ logits) sur les 42 coordonnées « main principale » : $\arg \max + \text{softmax} \rightarrow \text{letter} + \text{confidence}$.
6. Réponse JSON unique :

```
{
  "skeleton": [[x,y,vis], ... 9 pts],
  "hands": [[[x,y], ... 21 pts], ...],
  "letter": "A",
  "confidence": 0.84,
  "sign": null,
  "sign_confidence": null
}
```

Côté client

Le client dessine en permanence skeleton + hands. Pour la lettre :

- **Seuil de confiance** : on ignore la prédiction si `confidence < 0.6`.
- **Stabilité 10 frames** : on n'ajoute la lettre au mot que si la même valeur sort 10 frames de suite. Évite que le bruit entre deux signes pollue le mot final.
- **Aucun seuil géométrique** : la ligne du seuil (canvas, slider) est masquée en mode statique — l'inférence tourne sans interruption.

3 Mode dynamique — GRU Ocarina + déclencheur de seuil

Objectif : reconnaître les signes *en mouvement* (J, P, X, Z, mots complets) qui nécessitent une trajectoire temporelle.

Flux serveur

L'envoi reste continu (même rythme, même protocole). Les étapes 1-4 ci-dessus sont identiques. La différence vient après l'étape 4 :

5. Pour chaque session Socket.IO, le serveur maintient une **mémoire roulante** : un deque (`maxlen=45`) de vecteurs 42-float.
6. **Normalisation des landmarks** avant push dans le buffer :
 - rescale depuis la résolution runtime vers l'espace pixel d'entraînement (640×480),
 - recentre tous les points sur le poignet (point 0) → **invariance en translation**.
7. Toutes les `SIGN_EVERY_N = 5` frames, si le buffer est plein (45 frames disponibles), on appelle le **GRU Ocarina** sur la séquence complète.

Architecture GRU Ocarina. Embedding linéaire $42 \rightarrow 256 \rightarrow 64$ avec ReLU + Dropout 0,2, puis un GRU à 1 couche cachée (`hidden = 64`, `batch_first=True`), puis un Linear $64 \rightarrow N_{classes}$. On prend uniquement la sortie du dernier pas de temps. La prédiction sort sous forme `sign + sign_confidence`.

Réponse JSON enrichie :

```
{
  "skeleton": [...],
  "hands": [...],
  "letter": "A", // toujours calculé (gratuit)
  "confidence": 0.65,
  "sign": "Bonjour", // <- nouveau, GRU Ocarina
  "sign_confidence": 0.82
}
```

Le déclencheur de seuil côté client

C'est ici que le mode dynamique se distingue réellement : le client ne fait pas confiance à chaque `sign` reçu, il attend la **fin du geste** pour valider.

1. Le navigateur charge **MediaPipe Hands en local** (CDN, `modelComplexity=0`, ~ 12 MB cachés par le navigateur). Un RAF loop pompe la frame webcam locale et récupère les landmarks *côté client* indépendamment du serveur.
2. Une **ligne horizontale** est dessinée sur un canvas au-dessus de la vidéo, à la hauteur `thresholdRatio` ($\in [0,10 ; 0,90]$, réglable via un slider, persisté en `localStorage`).
3. À chaque tick RAF, on compare le *y* minimum des landmarks (point le plus haut) au seuil :

- **au-dessus** → ligne verte (« signe en cours »),
 - **en-dessous** pendant plus de 500 ms continus → ligne rouge (« en attente »).
4. À chaque prédiction GRU reçue du serveur avec **sign_confidence** $\geq 0,7$, on **stocke** la prédiction dans **pendingSign** (en gardant le sign de plus haute confiance si plusieurs arrivent).
 5. Au moment de la **transition au-dessus** → **en-dessous** (débounce 500 ms) : le client considère le geste terminé et **commit pendingSign** dans le mot.

Pas de stabilité 10 frames. Contrairement au MLP, le GRU travaille déjà sur 45 frames temporellement cohérentes. Il sort une prédiction déjà stable par construction. Imposer une stabilité supplémentaire reviendrait à sommer du bruit.

Le squelette reste affiché en permanence. Même quand la main est sous le seuil, le serveur continue de renvoyer **skeleton + hands** et le client les redessine. Seul le *commit du sign dans le mot* est gâté par le seuil.

4 Synthèse visuelle des deux flows

Statique

Dynamique

5 Récapitulatif des constantes

Constante	Valeur	Effet
SEQ_LEN	45	taille du buffer temporel GRU
SIGN_EVERY_N	5	inférence GRU 1 frame sur 5
TRAIN_W, TRAIN_H	640, 480	espace pixel canonique de normalisation
MIN_LETTER_CONFIDENCE	0,6	seuil de confiance MLP
STABLE_FRAMES_TO_COMMIT	10	stabilité MLP avant commit
MIN_SIGN_CONFIDENCE	0,7	seuil de confiance GRU
ABSENCE_GRACE_MS	500 ms	débounce de la descente sous le seuil
thresholdRatio	0,40	hauteur du seuil (slider 10–90 %)

Code source : <https://github.com/nmqx/hearmyhands>

Live : <https://hearmyhands.asia>